GraphCompose

v2.1.0

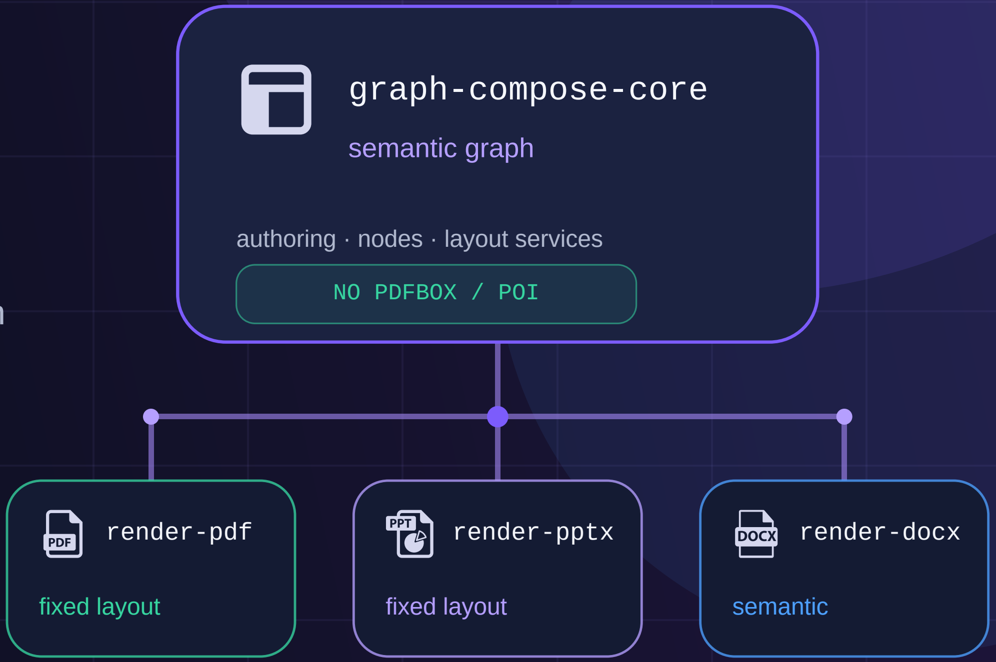
GRAPHCOMPOSE 2.1 / MODULE-FIRST

Compose once.
Render through modules.

A lean Java document core with a semantic model, deterministic layout geometry and opt-in output backends.

`io.github.demchaav:graph-compose-core`
+ choose a renderer module

THE 2.1 MODULE GRAPH

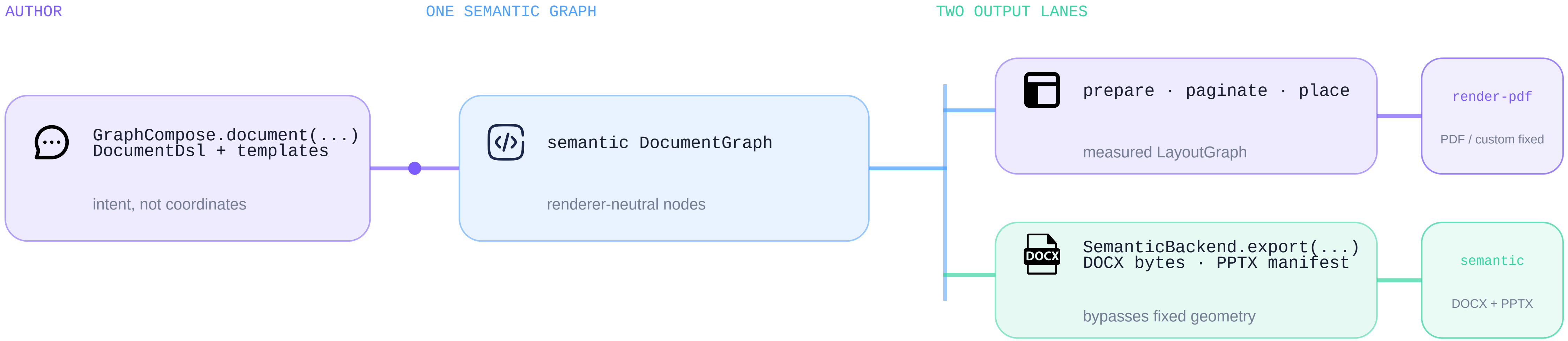


- `graph-compose-core` — lean engine
- `graph-compose` — drop-in PDF
- `ServiceLoader SPI` — bring a renderer
- `layoutSnapshot()` — geometry regression

01 / ARCHITECTURE

Semantic at the core. Format at the edge.

The canonical DSL produces one semantic DocumentGraph. Fixed-layout output resolves geometry; semantic exporters consume the graph directly.



Deterministic geometry is testable before bytes are rendered through layoutSnapshot().

LEAN CORE
No PDFBox, POI, zxing or template families in graph-compose-core.

FIXED BACKEND SPI
FixedLayoutBackendProvider and FontMetricsProvider are discovered by ServiceLoader.

SEMANTIC EXPORT
DOCX is a working POI exporter; PPTX is a slide-safe manifest skeleton in 2.0.

The renderer boundary is explicit; Android or another backend is an extension opportunity, not a shipped 2.0 claim.

02 / PACKAGING

Choose only what you need.

Eight engine-train artifacts move in lockstep. Fonts and emoji keep independent version lines; heavy render and template dependencies are opt-in.

DROP-IN PDF DEFAULT

graph-compose

Core + PDF renderer. Canonical callers keeping this coordinate retain PDF out of the box.

LEAN / CUSTOM LEAN

graph-compose-core

Authoring, semantic model and layout services. Add a renderer module only when needed.

BATTERIES INCLUDED BUNDLE

graph-compose-bundle

Default PDF stack plus templates, bundled fonts and emoji. Office exporters stay opt-in.

THE LOCKSTEP 2.0 TRAIN

One version, smaller dependency trees, explicit format boundaries.

- core — authoring + model
- render-pdf — fixed PDF
- render-docx — semantic DOCX
- render-pptx — manifest only
- templates — opt-in presets
- testing — snapshot tools
- graph-compose — PDF wrapper
- bundle — PDF + templates + assets

Accepted migration trade-off: built-in templates are no longer carried by graph-compose; add graph-compose-templates or use the bundle.

03 / REFERENCE RUN

Measured locally. Read with context.

Equivalent 1000-row report fixtures from the bundled 2026-06-15 single-machine snapshot. These values are directional, not a final 2.0 marketing benchmark.

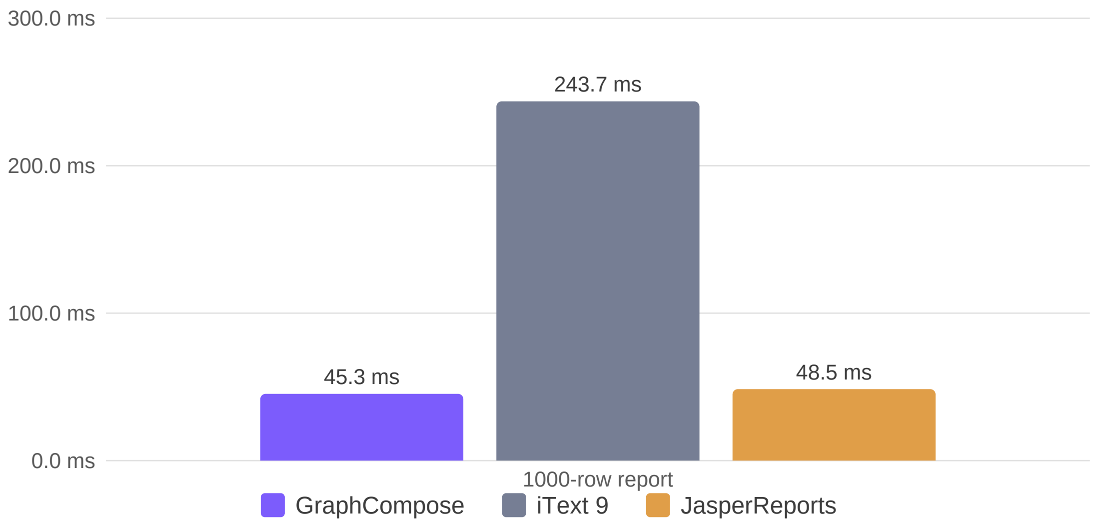
45.3 ms
GraphCompose avg render
1000 rows

15.7 MiB
avg allocated / render
thread allocation

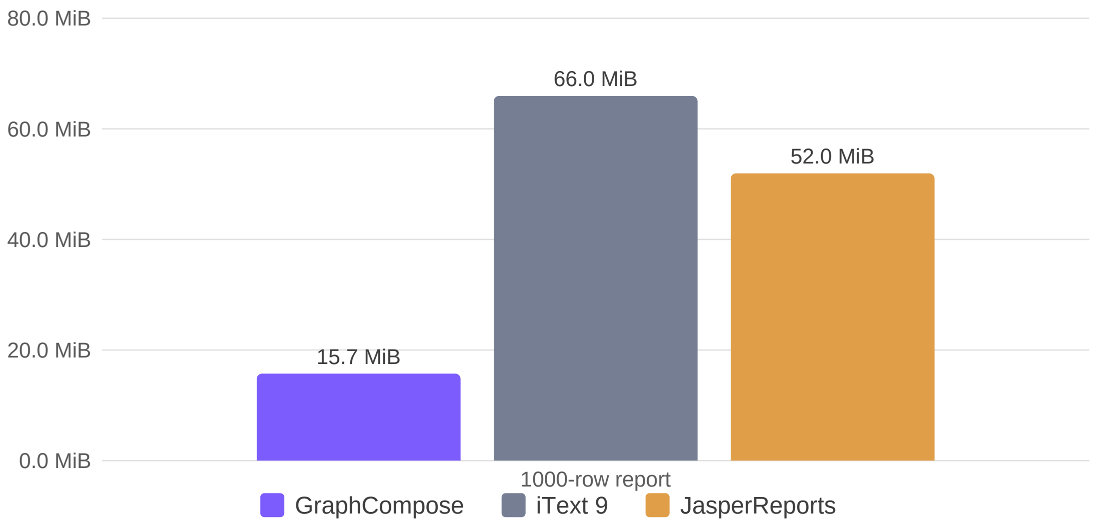
40 / 200 / 1000
report rows
scaling fixtures

2026-06-15
snapshot date
provenance incomplete

Average render latency at 1000 rows - ms



Average thread allocation per render - MiB



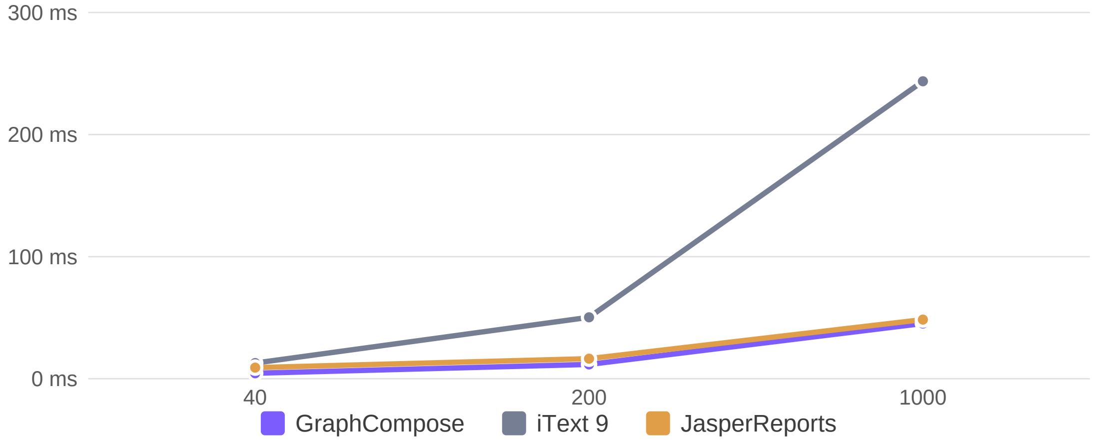
Scope: one developer machine; CPU, OS, JDK and commit were not stored in this historical file. The JSON stores 50/100 only for its small-invoice tier and omits scaling iteration metadata. Refresh both data and provenance on the final 2.0 commit before release claims.

Reproduce: scripts/run-benchmarks.ps1 - Current docs: docs/operations/benchmarks.md

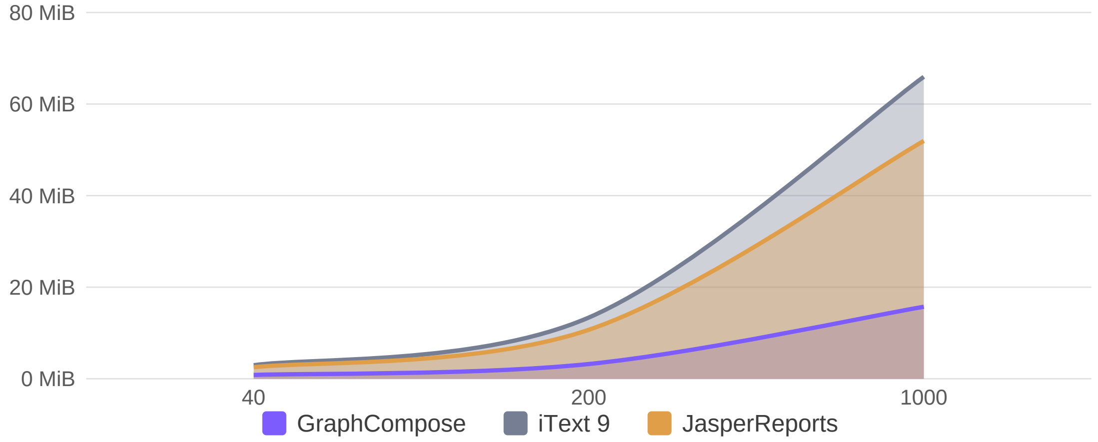
Watch the curve, not one headline number.

The same local reference run across 40, 200 and 1000 rows. Allocation values are independent per-render measurements; they are not shares of a common heap.

Average render latency vs. report size - ms



Average allocation vs. report size - MiB / render



5.38x
iText 9 took this many times longer in the 1000-row fixture

4.19x
iText 9 allocated this many times more in the same fixture

6.6%
latency gap vs JasperReports - inside the documented 5-10% local noise band

What 2.0 can claim today: **modular packaging, renderer isolation and deterministic layout snapshots**. Refresh performance evidence separately on the release commit.

GraphCompose renders this deck itself: gradients, canvas layers, SVG, clipping and native charts.